

Lesson 9 Structured Note: Parenthesis Matching as a Stack Application

0. Current Position in the Course

You are now starting **Lesson 9** of the stack curriculum.

Before Lesson 9, you completed two major phases:

Phase	Lessons	Main Focus
Phase 1	Lessons 1–4	Stack basics and array stack implementation
Phase 2	Lessons 5–8	Linked-list stack implementation in C and C++

So far, you have learned **how to build stacks**.

Now Lesson 9 starts a new idea:

How do we use stacks to solve real problems?

Lesson 9 belongs to:

Phase 3: Parenthesis and Bracket Matching

In this phase, stacks are used to check whether brackets in an expression are balanced.

Lesson 9 focuses only on **round parentheses**:

(and)

Later lessons extend this idea to:

(), [], {}

1. Lesson 9 Goal

By the end of this lesson, you should understand:

- What parenthesis matching means.
 - Why stacks are useful for parenthesis matching.
 - How to scan an expression from left to right.
 - Why opening parentheses are pushed onto the stack.
 - Why closing parentheses cause a pop operation.
 - How to detect extra opening parentheses.
 - How to detect extra closing parentheses.
 - Why non-parenthesis characters are ignored.
 - Why the final stack must be empty.
 - What this algorithm can check.
 - What this algorithm cannot check.
 - The time and space complexity of parenthesis matching.
-

2. Quick Review: What Is a Stack?

A stack is a data structure that follows this rule:

LIFO = Last In, First Out

This means:

The last item inserted is the first item removed.

Example:

```
push(10)
push(20)
push(30)
```

Logical stack view:

```
Top -> 30
       20
       10
```

If we call:

```
pop()
```

The removed value is:

```
30
```

Why?

Because `30` was inserted last, so it must be removed first.

This same idea helps us with nested parentheses.

3. What Problem Are We Solving?

We want to check whether parentheses in an expression are **balanced**.

Balanced means:

Every opening parenthesis `(` has a matching closing parenthesis `)` in the correct order.

Example of a balanced expression:

```
(A + B)
```

This is balanced because the opening parenthesis has a matching closing parenthesis.

Example of an unbalanced expression:

```
((A + B)
```

This is not balanced because one opening parenthesis is not closed.

Another unbalanced expression:

```
(A + B))
```

This is not balanced because there is an extra closing parenthesis.

4. What Does Balanced Mean?

An expression is balanced when all of these are true:

1. Every `(` has a matching `)`.
2. Every `)` has a previous unmatched `(`.
3. Parentheses close in the correct order.
4. No opening parenthesis is left over at the end.
5. No closing parenthesis appears too early.

Example:

```
((A+B) * (C-D))
```

This is balanced.

Why?

The parentheses can be paired like this:

```
( (A+B) * (C-D) )
^       ^       ^
|       |       |
outer  inner  inner
```

More simply:

```
(( ) ( ) )
```

Each opening parenthesis gets closed properly.

5. Why Do We Use a Stack?

Nested parentheses naturally follow the stack rule.

Look at this expression:

```
((A + B) * (C - D))
```

When we scan from left to right, we see:

```
(  
(  
A + B  
)  
*  
(  
C - D  
)  
)
```

The second (is opened after the first (.

So the second (must close before the first one closes.

That is exactly:

Last In, First Out

The most recently opened parenthesis must be closed first.

That is why a stack is a natural fit.

6. Simple Stack Idea for Parentheses

Think of every opening parenthesis (as saying:

I am waiting for a matching closing parenthesis.

So when we see:

(

We push it onto the stack.

When we see:

)

We need to match it with the most recent unmatched (.

So we pop one (from the stack.

The basic idea is:

```
Opening parenthesis '(' -> push
Closing parenthesis ')' -> pop
```

7. Characters We Care About

In Lesson 9, we only care about these two characters:

```
(
)
```

Everything else is ignored.

Ignored characters include:

```
A, B, C, D
numbers
+
-
*
/
spaces
```

Example:

```
(A + B) * (C - D)
```

The algorithm only pays attention to:

```
( ) ( )
```

It ignores:

```
A + B * C - D
```

8. Main Rule of the Algorithm

Scan the expression from left to right.

For each character:

```
If the character is '(':  
    push it onto the stack  
  
If the character is ')':  
    first check whether the stack is empty  
    if the stack is empty, return not balanced  
    otherwise, pop one '('  
  
If the character is anything else:  
    ignore it
```

At the end:

```
If the stack is empty:  
    the expression is balanced  
  
If the stack is not empty:  
    the expression is not balanced
```

9. Why Check Empty Before Popping?

When we see a closing parenthesis `)`, we want to match it with an earlier opening parenthesis `(`.

That means we need to pop one `(` from the stack.

But if the stack is already empty, there is no opening parenthesis available.

Example:

```
A + B)
```

There is a closing parenthesis, but no earlier opening parenthesis.

So this is not balanced.

This is similar to stack underflow:

Trying to pop from an empty stack

So the algorithm must check:

Is the stack empty before popping?

If yes, the expression is immediately not balanced.

10. Error Type 1: Extra Opening Parenthesis

An extra opening parenthesis means:

There is a (that never gets closed.

Example:

((A + B)

Only one closing parenthesis exists, but there are two opening parentheses.

The stack will still contain one (at the end.

So the result is:

Not balanced

This case is detected at the end of the scan.

Final check:

If stack is not empty:
not balanced

11. Error Type 2: Extra Closing Parenthesis

An extra closing parenthesis means:

There is a `)` that has no earlier unmatched `(`.

Example:

```
(A + B))
```

The first `)` matches the opening `(`.

Then the second `)` appears, but the stack is already empty.

So the result is:

```
Not balanced
```

This case is detected immediately while scanning.

Condition:

```
If current character is ')' and stack is empty:  
    not balanced
```

12. Balanced Case Dry Run

Expression:

```
(A + B)
```

Initial stack:

```
empty
```

Step-by-step trace:

Step	Character	Action	Stack After Action
1	(	push	(
2	A	ignore	(
3	+	ignore	(
4	B	ignore	(
5	)	pop	empty

At the end:

Stack is empty

So the expression is:

Balanced

13. Extra Opening Parenthesis Dry Run

Expression:

((A + B)

Initial stack:

empty

Step-by-step trace:

Step	Character	Action	Stack After Action
1	(	push	(
2	(	push	((
3	A	ignore	((
4	+	ignore	((
5	B	ignore	((

Step	Character	Action	Stack After Action
6	)	pop	(

At the end:

Stack still contains one '('

So the expression is:

Not balanced

Reason:

One opening parenthesis was never closed.

14. Extra Closing Parenthesis Dry Run

Expression:

(A + B))

Initial stack:

empty

Step-by-step trace:

Step	Character	Action	Stack After Action
1	(	push	(
2	A	ignore	(
3	+	ignore	(
4	B	ignore	(
5	)	pop	empty
6	)	stack is empty, cannot pop	error

At step 6, the algorithm sees `)`.

It tries to find a matching `(`.

But the stack is empty.

So the expression is:

Not balanced

Reason:

There is an extra closing parenthesis.

15. Full Required Dry Run

Expression:

`((A+B) * (C-D))`

Only parentheses matter:

`( ( ) ( ) )`

Initial stack:

empty

Trace:

Step	Character	Action	Stack After Action
1	<code>(</code>	push	<code>(</code>
2	<code>(</code>	push	<code>((</code>
3	<code>A+B</code>	ignore	<code>((</code>
4	<code>)</code>	pop	<code>(</code>

Step	Character	Action	Stack After Action
5	*	ignore	(
6	(	push	((
7	C-D	ignore	((
8	)	pop	(
9	)	pop	empty

Final stack:

empty

Final result:

Balanced

Why?

Every opening parenthesis got a matching closing parenthesis.

16. Algorithm in Plain English

Create an empty stack.

Scan the expression from left to right.

For every character:

If the character is '(':
Push it onto the stack.

Else if the character is ')':
If the stack is empty:
Return false.
Otherwise:
Pop one opening parenthesis.

Else:
Ignore the character.

After scanning the full expression:

```
If the stack is empty:  
    Return true.  
Otherwise:  
    Return false.
```

17. Algorithm as Pseudocode

```
isBalanced(expression):  
  
    create an empty stack  
  
    for each character ch in expression:  
  
        if ch == '(':  
            push(ch)  
  
        else if ch == ')':  
            if stack is empty:  
                return false  
            else:  
                pop()  
  
    if stack is empty:  
        return true  
    else:  
        return false
```

18. Important Final Check

The final stack check is necessary.

Suppose the expression is:

```
((A+B)
```

During scanning, no extra closing parenthesis appears.

So the algorithm does not fail early.

But after scanning, one `(` is still in the stack.

That means:

There is an opening parenthesis without a closing parenthesis.

So the final check must be:

`return stack is empty`

If the stack is empty, balanced.

If the stack is not empty, not balanced.

19. C++ Beginner Version Using `std::stack`

This version uses the built-in C++ stack.

The goal is to understand the algorithm first, without worrying about implementing the stack manually.

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isBalanced(string exp) {
    stack<char> st;

    for (int i = 0; i < exp.length(); i++) {
        if (exp[i] == '(') {
            st.push(exp[i]);
        }
        else if (exp[i] == ')') {
            if (st.empty()) {
                return false;
            }
            st.pop();
        }
    }

    return st.empty();
}
```

```
int main() {
    string exp = "((A+B) * (C-D))";

    if (isBalanced(exp)) {
        cout << "Balanced" << endl;
    } else {
        cout << "Not Balanced" << endl;
    }

    return 0;
}
```

Expected output:

Balanced

20. Code Explanation

Header Files

```
#include <iostream>
#include <stack>
#include <string>
```

Meaning:

Header	Purpose
<code><iostream></code>	Used for <code>cout</code>
<code><stack></code>	Gives access to C++ built-in stack
<code><string></code>	Allows use of <code>string</code>

Function Header

```
bool isBalanced(string exp)
```

This function receives an expression and returns:

Return Value	Meaning
true	Parentheses are balanced
false	Parentheses are not balanced

Creating the Stack

```
stack<char> st;
```

This creates a stack that stores characters.

Why `char`?

Because parentheses are characters:

```
'(' and ')'
```

Scanning the String

```
for (int i = 0; i < exp.length(); i++)
```

This loop checks every character in the expression from left to right.

Pushing Opening Parentheses

```
if (exp[i] == '(') {  
    st.push(exp[i]);  
}
```

If the current character is `(`, we push it.

Simple meaning:

```
This opening parenthesis is waiting to be matched later.
```

Handling Closing Parentheses

```
else if (exp[i] == ')') {  
    if (st.empty()) {  
        return false;  
    }  
    st.pop();  
}
```

If the current character is `)`, we need a matching `(`.

So first we check:

```
st.empty()
```

If the stack is empty, there is no matching opening parenthesis.

So we return:

```
false
```

Otherwise, we pop one `(`.

Final Return

```
return st.empty();
```

This means:

```
If the stack is empty, return true.  
If the stack is not empty, return false.
```

This final return checks whether any opening parentheses are left over.

21. What the Algorithm Checks

This algorithm checks only one thing:

Are the parentheses balanced?

It checks whether:

- Every (has a matching).
- No) appears without a previous (.
- No (remains at the end.

22. What the Algorithm Does Not Check

This algorithm does **not** check whether the full mathematical expression is valid.

Example:

(A +) * (- D)

The parentheses are balanced:

() ()

So the algorithm says:

Balanced

But the expression is mathematically wrong because operands are missing.

So do not confuse these two questions:

Question	Answered by Lesson 9 Algorithm?
Are the parentheses balanced?	Yes
Is the full expression mathematically valid?	No

Important idea:

Balanced parentheses do not always mean a valid expression.

23. Example: Balanced Parentheses but Invalid Expression

Expression:

$(A+)*(B)$

Parentheses only:

$() ()$

So the parenthesis matching algorithm says:

Balanced

But the expression itself is not mathematically valid because:

$A+$

has a missing operand after $+$.

So Lesson 9 is only about bracket matching, not full syntax checking.

24. Time Complexity

Let:

$n = \text{length of the expression}$

The algorithm scans every character once.

If the expression has 20 characters, it checks 20 characters.

If the expression has 100 characters, it checks 100 characters.

So the time complexity is:

$O(n)$

Reason:

One left-to-right scan of the expression

25. Space Complexity

The stack stores opening parentheses.

Worst case:

(((((A+B

Here, many opening parentheses appear before any closing parenthesis.

So the stack may grow with the size of the expression.

Worst-case space complexity:

$O(n)$

Reason:

In the worst case, the stack may store many opening parentheses.

26. Operation Cost Inside the Algorithm

Each stack operation is simple:

Operation	Time Complexity	Reason
push	$O(1)$	Adds one parenthesis to the top
pop	$O(1)$	Removes one parenthesis from the top
empty check	$O(1)$	Checks whether the stack has elements

But the full algorithm is still:

$O(n)$

Why?

Because we must scan the full expression character by character.

27. Three Final Possibilities

At the end of the algorithm, there are three main cases:

Situation	Meaning	Result
Stack is empty at the end	Every (was matched	Balanced
Stack is not empty at the end	Extra opening (exists	Not balanced
Need to pop but stack is empty	Extra closing) exists	Not balanced

These are the most important cases in Lesson 9.

28. Practice Examples

Example 1

(A+B)

Parentheses:

()

Result:

Balanced

Example 2

((A+B)

Parentheses:

(()

Result:

Not balanced

Reason:

One opening parenthesis is left over.

Example 3

(A+B))

Parentheses:

())

Result:

Not balanced

Reason:

One closing parenthesis has no matching opening parenthesis.

Example 4

(A+)*(B)

Parentheses:

```
( ) ( )
```

Result from parenthesis algorithm:

```
Balanced
```

But the expression is mathematically invalid.

Reason:

```
The algorithm checks parentheses only, not expression grammar.
```

29. Common Beginner Mistakes

Mistake 1: Thinking balanced parentheses means valid math

Wrong idea:

```
If parentheses are balanced, the whole expression is valid.
```

Correct idea:

```
Balanced parentheses only means the brackets match.  
The expression may still be mathematically wrong.
```

Mistake 2: Forgetting to check empty before pop

Wrong idea:

```
When you see ')', just pop.
```

Correct idea:

```
When you see ')', first check whether the stack is empty.
```


Why?

Because if the stack is empty, there is no matching `(`.

Mistake 3: Forgetting the final empty check

Wrong idea:

If no error happened during scanning, the expression is balanced.

Correct idea:

After scanning, the stack must be empty.

Example:

`((A+B)`

No extra closing parenthesis appears, but one opening parenthesis remains.

So the expression is not balanced.

Mistake 4: Checking all characters as if they matter

Wrong idea:

Every character affects the stack.

Correct idea:

Only `'('` and `')'` affect the stack in Lesson 9.

Letters, numbers, operators, and spaces are ignored.

30. Lesson 9 Key Takeaways

The core idea is:

Opening parenthesis '(' means push.
Closing parenthesis ')' means pop.

The algorithm works because nested parentheses follow the stack rule:

Last In, First Out

A balanced expression must satisfy:

No extra opening parentheses.
No extra closing parentheses.
Stack empty at the end.

The algorithm checks:

Parenthesis balance

It does not check:

Full mathematical correctness

Time complexity:

$O(n)$

Space complexity:

$O(n)$

31. Final Summary

Lesson 9 is the first major stack application.

Before this lesson, you learned how stacks are implemented using arrays and linked lists.

Now you are using a stack to solve a real problem:

Checking whether parentheses are balanced in an expression.

The stack is useful because the most recent opening parenthesis must be matched first.

That is exactly the stack rule:

Last In, First Out

The full logic is:

```
Scan left to right.  
Push '('.  
On ')', check empty first.  
If empty, not balanced.  
Otherwise, pop.  
At the end, stack must be empty.
```

If the stack is empty at the end:

Balanced

If the stack is not empty at the end:

Not balanced

If a closing parenthesis appears when the stack is empty:

Not balanced

That is the complete idea of Lesson 9.

32. Quick Self-Test

Try to answer these without looking above.

Question 1

What should the algorithm do when it sees (?

Answer:

Push it onto the stack.

Question 2

What should the algorithm do when it sees `)`?

Answer:

Check if the stack is empty.
If empty, return not balanced.
Otherwise, pop one opening parenthesis.

Question 3

What does it mean if the stack is not empty at the end?

Answer:

There are extra opening parentheses.
The expression is not balanced.

Question 4

What does it mean if the stack is empty when we see `)`?

Answer:

There is an extra closing parenthesis.
The expression is not balanced.

Question 5

Does this algorithm check whether the full expression is mathematically valid?

Answer:

No.
It only checks whether parentheses are balanced.

33. Mini Practice Task

For each expression, decide:

1. What does the parenthesis matching algorithm say?
2. Is the expression mathematically valid?

Expression	Parenthesis Algorithm Result	Math Validity
<code>(A+B)</code>	Balanced	Valid
<code>((A+B)</code>	Not balanced	Invalid
<code>(A+B))</code>	Not balanced	Invalid
<code>(A+)*(B)</code>	Balanced	Invalid

Main lesson:

Parenthesis matching is useful, but it is not the same as full expression validation.